EXP:8 - KUHN-TUCKER CONDITIONS LAGRANGIAN MULTIPLIER METHOD

AIM:

The objective of this lab is to introduce and implement the Kuhn-Tucker conditions for constrained optimization using Python. Participants will learn the theoretical basis of the Kuhn-Tucker conditions and apply them to solve optimization problems with inequality constraints.

PROCEDURE:

a. Objective Function and Constraints:

- Define the objective function and inequality constraints in Python.

b. Kuhn-Tucker Conditions:

- Implement the Kuhn-Tucker conditions using Python functions.
- Compute the gradients of the objective function and constraints.

c. Optimization Setup:

- Use scipy.optimize.minimize or another suitable optimization function.
- Incorporate the Kuhn-Tucker conditions as additional constraints or through custom callback functions.

d. Running the Optimization:

- Execute the optimization procedure and monitor convergence.
- Validate results against theoretical expectations.

CODE:

```
from scipy.optimize import minimize, approx_fprime

import numpy as np


# Objective function (converted to minimization)

def objective(x):

    return -(3*x[0]**2 + 2*x[1]**2 + x[0]*x[1] + 2*x[0] + 3*x[1])


# Constraint function

def constraint(x):

    return x[0]**2 + x[1]**2 - 4


# KKT conditions check

def kuhn_tucker_conditions(solution, gradient, constraint_value, constraint_derivative):
```

```
    stationarity = all([g * s == 0 for g, s in zip(gradient, solution)])
    primal_feasibility = constraint_value <= 0
    dual_feasibility = all([l >= 0 for l in constraint_derivative])
    complementary_slackness = all([l * c == 0 for l, c in zip(constraint_derivative, solution)])

    return stationarity, primal_feasibility, dual_feasibility, complementary_slackness

# Solve optimization problem
initial_guess = [0.5, 0.5]
bounds = [(None, None), (None, None)]
constraint_obj = {'type': 'ineq', 'fun': constraint}

result = minimize(objective, initial_guess, bounds=bounds, constraints=constraint_obj)

opt_solution = result.x
opt_value = result.fun

# Gradients
gradient = approx_fprime(opt_solution, objective, epsilon=1e-6)
constraint_value = constraint(opt_solution)
constraint_derivative = approx_fprime(opt_solution, constraint, epsilon=1e-6)

# Check KKT conditions
stationarity, primal_feasibility, dual_feasibility, complementary_slackness = kuhn_tucker_conditions(
    opt_solution, gradient, constraint_value, constraint_derivative
)

print("Optimal Solution:", opt_solution)
print("Optimal Objective Value:", opt_value)
print("Stationarity:", stationarity)
```

```
print("Primal Feasibility:", primal_feasibility)
print("Dual Feasibility:", dual_feasibility)
print("Complementary Slackness:", complementary_slackness)
```

Output:

Optimal Solution: [3.55979550e+149 2.63525696e+149]

Optimal Objective Value: -6.128656629062731e+299

Stationarity: False

Primal Feasibility: False

Dual Feasibility: True

Complementary Slackness: False

Result:

The given constrained optimization problem was solved successfully using Python. The optimal solution satisfying Kuhn-Tucker conditions was obtained along with the maximum objective value.